# Is It Simplicity or Speed? A Controlled Test of Simplicity Bias

## 1. Introduction

The claim under test: *when several features can solve a task, neural networks preferentially learn the simpler ones.* If true, it helps explain shortcut learning, failure under distribution shift, and why identically trained models can fail together (Shah et al., 2020). If only partly true, people may be fixing the wrong cause.

"Preferentially learn" can mean three different things:

- **Learned first.** The simple feature is picked up earlier in training.
- **Relied on at the end.** The trained network still depends on the simple feature.
- **Used exclusively.** The network ignores the complex feature completely.

An experiment can support the first and contradict the third, so a study must say which it tests. The claim also has a causal reading: the network prefers a feature *because it is simple*. That is its weakest part, for three reasons.

1. "Simple" has no single meaning (linear separability, low dimension, low frequency, short description length, ease of optimization).
2. In the datasets that support the claim, the simple feature differs in more than complexity: it is usually also learned faster, has a larger margin, and sits on one input axis.
3. The evidence comes mostly from one family of networks trained with SGD-style optimizers.

This study does not try to prove the claim wrong. It looks for the conditions where the claim holds, weakens or reverses, and asks how much of the preference remains once learning speed and predictive strength are matched.

## 2. Literature Review

**Central claims and evidence.**

| Paper | Central claim | How "simple" is defined | Evidence | Limit relevant here |
|---|---|---|---|---|
| Shah et al. (2020) | Networks trained with SGD can rely on the simplest predictive feature and ignore more complex ones, even when those are equally or more predictive | Number of linear pieces needed to read the feature | Synthetic linear + slab data; MNIST-CIFAR stacked images; persistence when the simple feature is less predictive; ensembles and adversarial training do not remove it; proof for | One definition of simplicity; datasets do not separate simplicity from learning speed or margin |

| Paper | Central claim | How "simple" is defined | Evidence | Limit relevant here |
|---|---|---|---|---|
| | | | one-hidden-layer ReLU nets | |
| Morwani et al. (2023) | A one-hidden-layer network can depend on only a low-dimensional projection of its input | Low dimension | Theory for linearly separable data in the infinite-width limit; Imagenette, Waterbirds-Landbirds | Needs linear separability, one hidden layer and a specific regime |
| Rahaman et al. (2019) | ReLU networks fit low-frequency parts of a function before high-frequency parts | Low frequency | Fourier analysis; synthetic experiments | About order of learning, not which feature a trained network relies on |

**Other work that shapes the design.**

- Kalimeris et al. (2019): early in training, an SGD network is largely explained by a linear classifier. This supports a speed-based reading.
- Pezeshki et al. (2021): gradient starvation. A strongly predictive feature shrinks the gradient and starves the others. This is a predictiveness explanation, not a complexity one.
- Hermann and Lampinen (2020); Scimeca et al. (2022): which feature a network picks depends on how easy it is to extract and how predictive it is.
- Puli et al. (2023): shortcut use can arise from cross-entropy margin maximization alone.
- Teney et al. (2024), Tsoy and Konstantinov (2024), and a NeurIPS 2025 paper on Adam and SGD: simplicity bias depends on activation, data geometry and optimizer.
- Kirichenko et al. (2023): networks often encode a feature without using it, so "not used" and "not learned" must be measured separately.
- Chizat et al. (2019): lazy training, which is why width and output scale matter.

**Definitions of simplicity in the literature.**

| Idea of simple | How it is measured | Where it appears |
|---|---|---|
| Few linear pieces | Pieces needed (more slabs means more pieces) | Shah et al. |
| Low dimension | Rank of the learned first layer | Morwani et al. |
| Low frequency | Fourier spectrum of the target | Rahaman et al. |
| Easy to optimize | Steps to reach a loss threshold | Kalimeris et al.; this study |
| Computational complexity | Parity order, circuit depth | This study (planned) |

**Open questions.** (1) The papers establish that reliance on simple features occurs in the datasets and regimes they test. They suggest, but do not establish, that simplicity itself is the cause. (2) Different definitions may rank the same two features in opposite orders, and I know of no direct test of which one predicts the network's choice. (3) I did not find a study that does a speed-matched test of reliance in a minimal setting, but my search was limited, so I claim no novelty beyond that.

## 3. Research Question

**Primary question.** When two features predict the same label, and their learning speed and predictive strength are held equal, does the intrinsic complexity of a feature still decide which one a network relies on?

**Why this question.** "Is the simple feature learned first?" can be answered yes without saying why. This question has different outcomes under different explanations.

**Sub-questions.**

- *Exchange rate.* How much extra predictive strength or margin does a complex feature need to overtake a simple one?
- *Definition conflict.* When two ideas of "simple" rank a pair of features oppositely, which one predicts the network's choice?
- *Robustness.* Does the answer change with optimizer, architecture, starting weights or training length?

**Direct and total effect.** If complexity acts only by making a feature slower to learn, then speed is a *mediator*, not a confounder. The design therefore reports two numbers: the effect with speed matched (the direct effect of complexity) and the effect with speed free to vary (the total effect). The difference between them is the finding. The pilot (section 8) touches only the primary question.

## 4. Hypothesis and Null Hypothesis

**How reliance is scored.** Train on data where feature A (simple) and feature B (complex) agree. Test on a *conflict set* where they point to opposite labels. The **preference index (PI)** is the fraction of conflict-set predictions that follow A, minus 0.5. PI = +0.5 means pure reliance on A, 0 means no preference, -0.5 means pure reliance on B. The pilot did not build a conflict set; it used a stand-in, the *gap*: accuracy lost when A is shuffled minus accuracy lost when B is shuffled. The gap spans about the same range, so the same cut-offs apply.

**Null hypothesis (H0).** With learning speed, margin and label correlation of A and B matched, PI does not depend on the complexity gap between them: the mean PI lies inside an equivalence band of ±0.10 around zero for every complexity gap, optimizer and architecture tested.

**Hypothesis (H1).** The preference for the simple feature is mostly explained by speed and strength, not complexity. Because H1 agrees with H0 for small gaps, the study can show H1 wrong. Four predictions:

- **P1.** Unmatched, PI is at least +0.4, reproducing the literature.
- **P2.** Matched, the complexity effect shrinks by at least half, and PI stays inside ±0.10 for small gaps (R up to 3).

- **P3.** A leftover preference for A remains for large gaps (R of 5 or more), because those features need more capacity and steps even at matched thresholds.
- **P4.** When two ideas of simplicity disagree, neither alone predicts PI above chance, for both optimizers.

**What would count against H1.** PI above +0.10 at every matched gap, with the confidence interval excluding the band (a direct effect of complexity); or a large PI that vanishes under random input rotation (an axis-alignment effect).

**Statistics for the full study.** At least 10 seeds per cell, bootstrap 95% confidence intervals, an equivalence test (TOST) so that "no effect" can be claimed rather than only "not significant", and a Holm correction over planned comparisons. The pilot used 5 seeds and none of these.

### Alternative explanations

The network may not select the simple feature *because it is simple*. Seven rivals produce the same observation:

1. **Optimization speed.** The simple feature is learned first, the loss gradient shrinks, and the complex feature is never learned.
2. **Margin or size.** Larger margins or input values give larger gradients, so that feature wins by construction.
3. **Gradient starvation.** The feature more tightly tied to the label starves the other, whatever its complexity.
4. **Coordinate alignment.** Networks are not rotation invariant; a feature on one input axis is easy to find at initialization.
5. **Definition artifact.** "Linear versus slab" mixes linear separability, piece count and frequency.
6. **Learned but unused.** The network may encode the complex feature without using it at the output.
7. **Training protocol.** Short training, one optimizer, or a very wide (lazy) network may fix the apparent order without being a general property.

## 5. Proposed Experiment Suite

### Dataset construction

Each example has 20 numbers: feature A, feature B and 18 distractors drawn from a standard normal. Labels are -1 or +1, balanced.

- **Feature A (simple).** A = label × U(0.5, 1.5). The sign of A gives the class, and no point is closer than 0.5 to the boundary.
- **Feature B (R regions).** The B axis is cut into R regions of width *s*, side by side. Even-numbered regions are class +1, odd-numbered are class -1. Points come from the middle 70% of a region (15% left empty at each edge) and are centred on zero, so B's smallest distance to a boundary is 0.15 *s*. With R = 2, B is a linear feature with a smaller margin than A.
- Both features predict the label perfectly alone. Neither is made stronger by design.
- **Single-feature versions** (the other feature replaced by noise) measure how fast each feature is learned alone.

- **Size in the pilot.** 1,000 training points, 4,000 test points.

### Features controlled

Complexity of B (number of regions R; later, parity order and spread over rotated coordinates), learning speed (matched), margin and size of B (standardized in the full study), and agreement with the label (equal for both features in E1, varied alone in E2).

### What changes between experiments, and what remains fixed

| ID | Question | What varies | What stays fixed | Main readout | Pilot status |
|---|---|---|---|---|---|
| E0 | Does the setup reproduce simplicity bias, and can speed matching be done? | Nothing (A-alone, B-alone, unmatched grid) | Architecture, optimizer | Steps to threshold; PI unmatched | Done, using the gap instead of PI |
| E1 | Does complexity matter once speed is matched? | Complexity gap; matched or unmatched | Margin family, optimizer, width | PI matched minus unmatched | Slab family only; valid for R = 2, 3 |
| E2 | How much strength offsets complexity? | Complexity gap and B's margin or agreement | Optimizer, width | Curve where PI = 0 | Not run |
| E3 | Do ideas of simplicity conflict? | Pairs ranked oppositely by two ideas | Strength matched across the pair | PI per pair and idea | Not run |
| E4 | Does it depend on optimizer, architecture, regime or duration? | One factor at a time | Data, complexity gap | PI trajectory; probe accuracy | Optimizer only |
| E5 | Can a complex feature be learned first? | B strongly predictive, A weakened by label noise | Complexity of B | Learning-order trajectory | Not run |

**Fixed in every pilot run:** network (2 × 64 ReLU), 1,000 training points, 18 distractors, full batch, 2,000 steps, learning rate per optimizer. **Changed in the pilot:** R (2, 3, 5, 7), scale *s* (1 or tuned), optimizer (Adam or momentum GD).

### Model architecture

A multilayer perceptron in PyTorch: 20 inputs, two hidden layers of 64 ReLU units, one output, default initialization. The full study adds widths 256 and 1024, tanh, a Fourier-feature input, an output-scale parameter to move between rich and lazy regimes.

### Training procedure

Binary cross-entropy, full batch (all 1,000 points per step), 2,000 steps, no early stopping, no weight decay. Optimizers: Adam (learning rate 0.001) and "SGD" (learning rate 0.05, momentum 0.9). Because training is full batch, the second is gradient descent with momentum, not stochastic. 5 seeds per cell; data seed is 1000 + seed, network seed is the seed. The full study adds mini-batch plain SGD, a second learning rate per optimizer, and training far past 100% accuracy.

**Speed matching.** Speed is the first step at which training loss falls below 0.1 (2,000 if never). The scale $s$ is tuned by bisection on a log scale in [0.3, 30] (5 steps, 2 calibration seeds) until B alone reaches the threshold in as many steps as A alone. A matched cell is a *valid test* only if the speed ratio (steps for B divided by steps for A) is within 10% of 1 and test accuracy is at least 0.9.

### Evaluation methodology

Final accuracy cannot show which feature a model uses, since both give perfect accuracy. Several measures are used, each with a stated weakness.

| Measure | What it does | Weakness |
|---|---|---|
| Conflict-set PI (primary, full study) | Counts how often the network follows A when A and B disagree | Conflict examples never appear in training, so it describes the learned function, not natural data |
| Shuffle drop (used in the pilot) | Shuffles one feature across test examples and records the accuracy lost | Shuffled inputs are off-distribution |
| Trajectory every 50 steps (full study) | Records both measures through training | Costs extra compute |
| Linear probes on the last hidden layer (full study) | Reads out A and B from the hidden representation | A probe may read what the output does not use (that is the point, but it needs a random-label control) |

If a network used both features equally, each shuffle drop would be about 0.25. Features are shuffled rather than set to zero, because zero is not neutral: for B it can lie inside a region of one class.

**Cost.** Full suite about 1,500 to 2,000 runs, a few CPU-hours. The pilot grid took about 12 minutes on a Colab CPU.

## 6. Controls and Confounding Variables

Each row names a rival explanation, the control that isolates it, the check that it worked, and how far the pilot got.

| Rival explanation | Control | Check that it worked | In the pilot |
|---|---|---|---|
| Optimization speed | Match steps-to-threshold of A alone and B alone within 10% by tuning margin or agreement, not input size; regression fallback where matching fails | Loss curves of A-alone and B-alone overlap; speed ratio near 1 | Partly: matched by tuning *s*; valid only for R = 2, 3 |
| Margin or size | Standardize all inputs; report margin ratio and include it in the regression | Equal input variance; margin ratio near 1 | Not controlled: *s* changes B's size and margin together |
| Gradient starvation | Equal agreement probability for A and B in E1; vary it alone in E2 | PI as a function of agreement at fixed complexity | Not run (both features perfectly predictive) |
| Coordinate alignment | Random rotation of inputs; distributed-feature family | PI before and after rotation | Not run |
| Definition artifact | Three complexity families and contradictory pairs (E3) | Whether families give the same PI ordering | Slab family only |
| Learned but unused | Linear probes; last-layer retraining | High probe accuracy for B while PI favors A | Not run |
| Training protocol | Vary optimizer, width, activation, lazy scale, duration one at a time | PI stable or not across each factor | Optimizer only |
| Seed and data noise | At least 10 seeds, fresh data per seed, bootstrap intervals | Interval width against the ±0.10 band | 5 seeds, standard deviation only |
| Off-distribution test artifact | Report both conflict-set PI and in-distribution ablation drop | The two measures rank cells the same way | Ablation drop only |

**Baselines.** (a) A-alone and B-alone networks, showing what each feature costs in isolation. (b) Two linear features (R = 2), where any preference cannot be about complexity. (c) A random-label control for the probes. The pilot has (a) and (b), not (c).

**Regression fallback.** Across the full grid, regress PI on complexity gap, log speed ratio, margin ratio and correlation ratio. The coefficient on complexity is then the direct effect even where matching is imperfect.

The experiment that separates complexity from correlation strength is E2: fix complexity, vary only B's agreement with the label, and see where PI crosses zero.

## 7. Expected Outcomes and Interpretation

No outcome is assumed. The last column shows what the pilot has seen so far.

| Possible result | Interpretation | Pilot so far |
|---|---|---|
| PI large unmatched, near zero matched (my expectation) | The literature's effect is largely speed or margin; the claim needs qualifying | Fits Adam R = 2 (+0.47 to 0.00) and momentum GD R = 3 (+0.50 to -0.01); momentum GD R = 2 ends at +0.16 |
| PI stays above +0.10 at every matched gap, interval excluding the band | Complexity has a direct effect; the claim survives this test | Not seen, but only 3 valid cells and 5 seeds |
| PI shifts toward B when B is made stronger (E2) | Correlations can overpower simplicity; the exchange rate sizes the effect | Not run |
| Large PI for two linear features | The preference is margin, size or alignment, since complexity is identical | Yes: at R = 2 the network used only A (+0.47, +0.49) |
| PI differs between Adam and SGD | Optimization, not architecture, is responsible | Possible at R = 2 matched (0.00 vs +0.16); 5 seeds cannot settle it |
| PI disappears under random rotation | The effect was axis alignment | Not run |
| Different complexity families give different PI orderings (E3) | "Simplicity" is not one thing; the claim must name its definition | Not run |
| Probe accuracy for B high but PI favors A | The complex feature is learned but unused | Not run |
| PI drifts toward B with longer training | The preference is a transient of learning order | Not run (final model only) |
| B cannot be matched to A's speed | The question cannot be answered in this setup; report the limit | Yes: R = 5 and 7, both optimizers |
| No effect at any tested setting | A failure to falsify under these conditions; says nothing about natural images or large models | Not applicable yet |

A mixed result is most likely: a preference present in some settings and absent in others. Locating that boundary is the aim. If the results support the claim, I would report that I tried to falsify it and could not

under these named conditions, and I would not say simplicity bias is proved. A confirmation with strong controls is a finding, not a failure.

## 8. Optional Experimental Results (Pilot)

### Methodology

The pilot is the notebook (PyTorch, 5 seeds per cell) and uses the design in section 5 with the shuffle gap in place of the conflict-set PI. It differs from the full design as follows.

| Item | Full design | Pilot | Does it matter? |
|---|---|---|---|
| Network | 2 × 100 to 1024 units | 2 × 64 | Minor for a pilot |
| Data size | 10,000 train | 1,000 train, 4,000 test | Noisier estimates |
| Training | Mini-batch plain SGD | Full batch; "SGD" has momentum 0.9 | Yes: mini-batch noise is untested. A 2026 paper (Mirzaie et al., read only in summary) reports that the shortcut-reducing effect comes from mini-batch noise |
| Seeds, statistics | 10 seeds, 95% intervals, equivalence band | 5 seeds, standard deviation | Yes: no "no effect" claim is possible yet |
| Reliance measure | Conflict-set PI, probes | Shuffle gap only | Yes: gap is a stand-in for PI |
| Size and margin | Standardized, margin tuned | Not controlled; *s* changes both | Yes: the main confound |
| Checkpoints | Reliance at 8 points in training | Final model only | Yes: "learned first" cannot be separated from "relied on at the end" |
| Pre-set checks | Outcome table before running | Checks C1–C5; C3 failed | Good practice; C3 failing is a result |

### Results

Speed ratio is steps for B alone divided by steps for A alone to reach loss 0.1. A alone needed 64.5 steps with Adam and 36 with momentum GD. Values are means over 5 seeds.

| Optimizer | R | Condition | *s* | Speed ratio | Test acc | Drop (A shuffled) | Drop (B shuffled) | Gap (A − B) | Valid? |
|---|---|---|---|---|---|---|---|---|---|
| Adam | 2 | unmatched | 1.00 | 1.28 | 1.00 | 0.48 | 0.02 | +0.47 | yes |
| Adam | 2 | matched | 2.09 | 1.02 | 1.00 | 0.25 | 0.25 | 0.00 | yes |
| Adam | 3 | unmatched | 1.00 | 2.06 | 1.00 | 0.50 | 0.00 | +0.50 | yes |
| Adam | 3 | matched | 27.92 | 1.16 | 1.00 | 0.06 | 0.44 | -0.38 | no (speed) |
| Adam | 5 | unmatched | 1.00 | 3.05 | 1.00 | 0.50 | 0.00 | +0.50 | yes |
| Adam | 5 | matched | 27.92 | 2.68 | 1.00 | 0.50 | 0.00 | +0.49 | no (speed) |
| Adam | 7 | unmatched | 1.00 | 3.19 | 1.00 | 0.50 | 0.00 | +0.50 | yes |
| Adam | 7 | matched | 27.92 | 10.19 | 1.00 | 0.50 | 0.00 | +0.50 | no (speed) |
| SGD | 2 | unmatched | 1.00 | 1.58 | 1.00 | 0.50 | 0.00 | +0.49 | yes |
| SGD | 2 | matched | 1.81 | 1.04 | 1.00 | 0.33 | 0.17 | +0.16 | yes |
| SGD | 3 | unmatched | 1.00 | 4.22 | 1.00 | 0.50 | 0.00 | +0.50 | yes |
| SGD | 3 | matched | 4.96 | 0.99 | 1.00 | 0.25 | 0.26 | -0.01 | yes |
| SGD | 5 | unmatched | 1.00 | 9.26 | 1.00 | 0.50 | 0.00 | +0.50 | yes |
| SGD | 5 | matched | 27.92 | 55.56 | 0.70 | 0.20 | 0.00 | +0.20 | no (speed, training failed) |
| SGD | 7 | unmatched | 1.00 | 9.61 | 1.00 | 0.50 | 0.00 | +0.50 | yes |
| SGD | 7 | matched | 27.92 | 55.56 | 0.51 | 0.00 | 0.00 | 0.00 | no (speed, training failed) |

"SGD" is full-batch gradient descent with momentum. A speed ratio of 55.56 means B alone never reached the threshold in 2,000 steps. Standard deviations were 0.03 or less except SGD R = 5 matched (0.27, where training failed in some seeds). Notebook checks C1, C2, C4, C5 passed; C3 (matched R = 2: speed within 10% and more reliance on B than on A) did not.

### Analysis

- **Unmatched, the network used only A** for every R and both optimizers (shuffling A cost 0.48–0.50; shuffling B at most 0.02). This reproduces the literature, but the setup is confounded.
- **At R = 2 both features are linear, yet only A was used.** A has the larger margin (0.5 against 0.15) and is learned faster (B alone needs 1.28× as many steps with Adam, 1.58× with momentum GD). That preference cannot be about complexity.
- **Matching speed removed most of the gap, but not in a stable direction.** At R = 2, with speeds within 5%, the gap fell from about +0.47 to 0.00 (Adam) and +0.16 (momentum GD). A speed difference of only 28–58% gave an almost total preference, so the network is very sensitive to small speed differences.
- **At R = 3 with momentum GD,** a 3-region feature was not disfavoured once speed was matched (ratio 0.99, drops 0.25 and 0.26). The Adam R = 3 cell shows reliance on B but misses the speed criterion and sits near the scale ceiling, so I do not count it.
- **R = 5 and 7 gave no valid test.** B could not be made as fast as A within the scale range, and momentum GD was unstable at large scale. Speed is also not monotone in scale: for Adam at R = 7, B alone needed about 206 steps at $s$ = 1 and about 658 at $s$ = 27.9, so the bisection can fail silently.
- **Proportional reading.** None of the three valid matched cells reproduces the unmatched gap of about +0.5, and the largest remaining gap toward A is +0.16. This supports the weaker claim that most of the unmatched preference comes from speed, margin or both. It cannot say whether a small direct effect of complexity remains, because size is uncontrolled, only R up to 3 gave valid cells, and there are 5 seeds. A second NumPy implementation reported in my earlier write-up is not in the notebook, so I could not check it here.

## 9. Limitations

**What the experiment cannot establish.**

- Anything directly about natural images, language or large networks. Synthetic features are independent and axis-aligned; real features are neither.
- That complexity never matters. A null result is a failure to falsify under the tested conditions only.
- Training dynamics from the conflict set. It is off-distribution (no training example has A and B disagreeing), so it describes the learned function.

**Variables that may still be confounded.**

- *Speed as mediator.* If complexity makes a feature slow, matching speed removes part of complexity's effect. Reporting matched and unmatched results shows the split, but cannot say how much of the total counts as genuine simplicity bias.
- *Matching is approximate* (a threshold, a few seeds, up to 10% tolerance), and the regression fallback assumes a roughly linear model of the confounds.
- *Margin, agreement and speed are correlated,* so a clean single-variable change is not always possible.

**Whether the definition of simplicity is incomplete.** Yes. The suite covers linear pieces, parity order, distributed dimension and speed. It leaves out description length, Kolmogorov complexity, frequency on a data manifold, and any definition on learned representations. In the slab family, more regions also means higher frequency, so "fewer pieces" and "lower frequency" cannot be separated.

**Pilot limits.** 5 seeds, width 64, 1,000 training points, unstandardized inputs, no conflict set, no checkpoints; calibration failed for R of 5 or more; "SGD" is full batch with momentum; the shuffle drop is measured on shuffled inputs only.

**How future experiments could improve the investigation.**

1. Standardize inputs and tune B's margin instead of its scale.
2. Add the conflict-set PI and record it every 50 steps.
3. Use 10 or more seeds with bootstrap intervals and the equivalence test.
4. Widen or replace the matching knob so R = 5 and 7 can be matched, or report that they cannot.
5. Add mini-batch SGD without momentum and a second learning rate per optimizer.
6. Run E2 and E3 (exchange rate, conflicting definitions), then move to image-based features, convolutional nets and transformers.

## 10. Conclusion

This study cannot show that simplicity bias is absent, and is not built to. It is built to say how much of the preference for simple features comes from complexity itself, as opposed to learning speed, margin, correlation strength and axis alignment, and under which optimizers and architectures that share changes.

Three outcomes would be informative. If the preference shrinks a lot once speed and strength are matched, the claim should be restated as one about ease of optimization. If it survives matching and rotation, simplicity bias has a direct component, and the exchange rate from E2 measures its size. If different ideas of simplicity give different answers, the claim needs a stated definition before it can be tested.

The pilot already shows why this matters. Unmatched, a network preferred one linear feature over another, which cannot be about complexity. Once speed was matched, the gap fell from about +0.5 to between -0.01 and +0.16 in the three valid cells. That supports the weaker claim that most of the preference comes from speed or margin, and leaves open whether a small direct effect remains. "I tried to falsify the claim under these conditions and could not" is a valid outcome of this design.

## References


1. Shah, H., Tamuly, K., Raghunathan, A., Jain, P., and Netrapalli, P. (2020). The Pitfalls of Simplicity Bias in Neural Networks. NeurIPS 2020. arXiv:2006.07710
2. Morwani, D., Batra, J., Jain, P., and Netrapalli, P. (2023). Simplicity Bias in 1-Hidden Layer Neural Networks. NeurIPS 2023. arXiv:2302.00457
3. Rahaman, N., et al. (2019). On the Spectral Bias of Neural Networks. ICML 2019. arXiv:1806.08734
4. Kalimeris, D., Kaplun, G., Nakkiran, P., Edelman, B., Yang, T., Barak, B., and Zhang, H. (2019). SGD on Neural Networks Learns Functions of Increasing Complexity. NeurIPS 2019. arXiv:1905.11604

5. Pezeshki, M., et al. (2021). Gradient Starvation: A Learning Proclivity in Neural Networks. NeurIPS 2021. arXiv:2011.09468
6. Hermann, K. L., and Lampinen, A. K. (2020). What Shapes Feature Representations? NeurIPS 2020. arXiv:2006.12433
7. Scimeca, L., et al. (2022). Which Shortcut Cues Will DNNs Choose? ICLR 2022. arXiv:2110.03095
8. Puli, A. M., Zhang, L. H., Wald, Y., and Ranganath, R. (2023). Don't Blame Dataset Shift! Shortcut Learning due to Gradients and Cross Entropy. NeurIPS 2023. arXiv:2308.12553
9. Teney, D., et al. (2024). Neural Redshift: Random Networks are not Random Functions. arXiv:2403.02241
10. Tsoy, N., and Konstantinov, N. (2024). Simplicity Bias of Two-Layer Networks beyond Linearly Separable Data. arXiv:2405.17299
11. The Rich and the Simple: On the Implicit Bias of Adam and SGD. NeurIPS 2025. arXiv:2505.24022 (authors not recorded here)
12. Mirzaie, N., et al. (2026). Implicit Regularization of SGD Reduces Shortcut Learning. ICLR 2026. (Read only in summary; not verified.)
13. Kirichenko, P., Izmailov, P., and Wilson, A. G. (2023). Last Layer Re-Training is Sufficient for Robustness to Spurious Correlations. ICLR 2023.
14. Qiu, G., Kuang, D., and Goel, S. (2024). Complexity Matters: Feature Learning in the Presence of Spurious Correlations. ICML 2024. (Not checked.)
15. Teney, D., Abbasnejad, E., Lucey, S., and van den Hengel, A. (2022). Evading the Simplicity Bias. CVPR 2022.
16. Chizat, L., Oyallon, E., and Bach, F. (2019). On Lazy Training in Differentiable Programming. NeurIPS 2019.